Django Complete Notes — Beginner to Advanced

Table of Contents

1. Django Introduction
2. Django Installation and Setup
3. Django Project Structure
4. URL Dispatcher
5. Function-Based Views (FBV)
6. Class-Based Views (CBV)
7. Django Templates
8. Template Inheritance
9. Static Files
10. Models and Database
11. Django ORM
12. CRUD Operations
13. Forms and ModelForms
14. Django Admin
15. Authentication and Authorization
16. Sessions and Cookies
17. Middleware
18. File Uploads
19. Pagination
20. Messages Framework
21. Signals
22. Generic Class-Based Views
23. Query Optimization
24. Relationships Between Models
25. Django REST Framework
26. API Authentication and Permissions
27. Security Best Practices
28. Testing
29. Deployment
30. Interview Questions

31. Practice Projects

---

1. Django Introduction

What is Django?

Django is a high-level Python web framework used to build secure, maintainable, and scalable web applications quickly.

Features

- MVT architecture
- Built-in ORM
- Authentication system
- Admin panel
- URL routing
- Template engine
- Form handling
- Security features
- Database migrations
- Testing support

MVT Architecture

- **Model:** Handles data and database operations.
- **View:** Handles request processing and business logic.
- **Template:** Displays the user interface.

Request flow:

Browser → URL Dispatcher → View → Model → Database → View → Template → Response

---

2. Django Installation and Setup

Install Django

```
python -m venv venv
```

Activate the virtual environment on Windows:

```
venv\Scripts\activate
```

Install Django:

```
pip install django
```

Check the version:

```
python -m django --version
```

Create a Project

```
django-admin startproject config .
```

Create an App

```
python manage.py startapp core
```

Add the app to `config/settings.py`:

```
INSTALLED_APPS = [
    # Default Django apps
    "core",
]
```

Run the development server:

```
python manage.py runserver
```

Open:

```
http://127.0.0.1:8000/
```

Common Commands

```
python manage.py check
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py showmigrations
python manage.py shell
python manage.py test
```

---

3. Django Project Structure

```
project/
│
├── manage.py
├── config/
```

```
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
└── core/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── views.py
    └── migrations/
```

Important Files

- `manage.py` : Project management commands.
- `settings.py` : Project configuration.
- `urls.py` : URL routing.
- `views.py` : Request-handling logic.
- `models.py` : Database models.
- `admin.py` : Admin registration.
- `tests.py` : Automated tests.
- `migrations/` : Database schema changes.

---

4. URL Dispatcher

What is URL Dispatcher?

Django's URL dispatcher maps a requested URL to the appropriate view function or class.

Example

`core/views.py`

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Home Page")
```

```
  def about(request):
      return HttpResponse("About Page")
```

`core/urls.py`

```
  from django.urls import path
  from . import views

  urlpatterns = [
      path("", views.home, name="home"),
      path("about/", views.about, name="about"),
  ]
```

`config/urls.py`

```
  from django.contrib import admin
  from django.urls import path, include

  urlpatterns = [
      path("admin/", admin.site.urls),
      path("", include("core.urls")),
  ]
```

URL Parameters

Integer Parameter

```
  path("user/<int:user_id>/", views.user_detail, name="user_detail")
```

```
  def user_detail(request, user_id):
      return HttpResponse(f"User ID: {user_id}")
```

String Parameter

```
  path("category/<str:name>/", views.category, name="category")
```

Slug Parameter

```
  path("post/<slug:slug>/", views.post_detail, name="post_detail")
```

UUID Parameter

```
  path("order/<uuid:order_id>/", views.order_detail, name="order_detail")
```

Built-in URL Converters

- `str`: Non-empty string excluding `/`.

- `int` : Positive integer including zero.
- `slug` : Letters, numbers, hyphens, and underscores.
- `uuid` : UUID value.
- `path` : String that may contain `/` .

Named URLs

```
path("contact/", views.contact, name="contact")
```

Reverse a URL in Python:

```
from django.urls import reverse

url = reverse("contact")
```

Reverse a URL in a template:

```
<a href="{% url 'contact' %}">Contact</a>
```

---

5. Function-Based Views (FBV)

What is FBV?

A Function-Based View is a Python function that receives an HTTP request and returns an HTTP response.

```
from django.http import HttpResponse


def home(request):
    return HttpResponse("Hello Django")
```

Render a Template

```
from django.shortcuts import render


def home(request):
    context = {
        "name": "Faruk",
        "language": "Python",
    }

    return render(request, "home.html", context)
```

Redirect to Another URL

```
from django.shortcuts import redirect


def go_home(request):
    return redirect("home")
```

Get an Object or Return 404

```
from django.shortcuts import get_object_or_404
from .models import Post


def post_detail(request, pk):
    post = get_object_or_404(Post, pk=pk)

    return render(
        request,
        "posts/detail.html",
        {"post": post},
    )
```

Handle GET and POST

```
from django.shortcuts import render, redirect


def contact(request):
    if request.method == "POST":
        name = request.POST.get("name", "")

        # Process the submitted data here.

        return redirect("contact")

    return render(request, "contact.html")
```

FBV Advantages

- Easy to understand.
- Suitable for custom business logic.
- Straightforward for beginners.
- Flexible request handling.

---

6. Class-Based Views (CBV)

What is CBV?

A Class-Based View uses a Python class to handle HTTP requests through methods such as `get()` and `post()`.

```
from django.http import HttpResponse
from django.views import View


class HomeView(View):
    def get(self, request):
        return HttpResponse("Home Page")
```

URL configuration:

```
from django.urls import path
from .views import HomeView

urlpatterns = [
    path("", HomeView.as_view(), name="home"),
]
```

TemplateView

```
from django.views.generic import TemplateView


class AboutView(TemplateView):
    template_name = "about.html"
```

```
path("about/", AboutView.as_view(), name="about")
```

Advantages

- Encourages reusable code.
- Supports inheritance and mixins.
- Reduces repetition.
- Integrates with generic views.

---

7. Django Templates

Templates generate HTML using context data and Django Template Language (DTL).

Variable

```
<h1>Hello {{ name }}</h1>
```

If Statement

```
{% if user.is_authenticated %}
    <p>Welcome, {{ user.username }}</p>
{% else %}
    <p>Please log in.</p>
{% endif %}
```

For Loop

```
{% for post in posts %}
    <h2>{{ post.title }}</h2>
    <p>{{ post.content }}</p>
{% empty %}
    <p>No posts available.</p>
{% endfor %}
```

Filters

```
{{ name|lower }}
{{ name|upper }}
{{ title|title }}
{{ content|truncatewords:20 }}
{{ created_at|date:"Y-m-d" }}
{{ items|length }}
```

Comments

```
{# This is a template comment #}
```

Template Configuration

In `settings.py`:

```
TEMPLATES = [
    {
        "BACKEND": "django.template.backends.django.DjangoTemplates",
        "DIRS": [BASE_DIR / "templates"],
        "APP_DIRS": True,
        "OPTIONS": {
```

```
            "context_processors": [
                "django.template.context_processors.request",
                "django.contrib.auth.context_processors.auth",
                "django.contrib.messages.context_processors.messages",
            ],
        },
    },
]
```

Create a project-level `templates/` directory.

---

8. Template Inheritance

Template inheritance avoids repeating common HTML structures.

Base Template

`templates/base.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}My Website{% endblock %}</title>
</head>
<body>
    <header>
        <h1>My Website</h1>
    </header>

    <main>
        {% block content %}
        {% endblock %}
    </main>
</body>
</html>
```

Child Template

`templates/home.html`

```
{% extends "base.html" %}

{% block title %}Home{% endblock %}
```

```
{% block content %}
    <h2>Welcome to Django</h2>
{% endblock %}
```

---

9. Static Files

Static files include CSS, JavaScript, and images used by the website.

Settings

```
STATIC_URL = "static/"
```

Template

```
{% load static %}

<link rel="stylesheet" href="{% static 'css/style.css' %}">
<script src="{% static 'js/app.js' %}" defer></script>
```

Recommended structure:

```
core/
└── static/
    └── core/
        ├── css/
        │   └── style.css
        ├── js/
        │   └── app.js
        └── images/
```

Production static-file collection:

```
python manage.py collectstatic
```

Configure `STATIC_ROOT` for production deployment.

---

10. Django Models

A model represents a database table.

Create a Model

`core/models.py`

```
from django.db import models


```

```
class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    published = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return self.title
```

Common Model Fields

- `CharField`: Short text.
- `TextField`: Long text.
- `IntegerField`: Integer.
- `FloatField`: Floating-point number.
- `DecimalField`: Fixed-precision decimal.
- `BooleanField`: True or false.
- `DateField`: Date.
- `DateTimeField`: Date and time.
- `EmailField`: Email string validation.
- `URLField`: URL string validation.
- `FileField`: File path.
- `ImageField`: Image path.
- `ForeignKey`: Many-to-one relationship.
- `OneToOneField`: One-to-one relationship.
- `ManyToManyField`: Many-to-many relationship.
- `UUIDField`: UUID value.

Model Options

```
class Meta:
    ordering = ["-created_at"]
    verbose_name = "Post"
    verbose_name_plural = "Posts"
```

Migrations

```
python manage.py makemigrations
python manage.py migrate
```

```
python manage.py showmigrations
```

Migrations keep Django's model definitions synchronized with the database schema.

---

11. Django ORM

ORM stands for Object-Relational Mapper.

It lets developers interact with the database using Python objects rather than writing SQL for every operation.

Create

```
Post.objects.create(
    title="Django ORM",
    content="Learning Django",
    published=True,
)
```

Read All

```
posts = Post.objects.all()
```

Get One

```
post = Post.objects.get(id=1)
```

`get()` raises `DoesNotExist` if no record matches and `MultipleObjectsReturned` if multiple records match.

Filter

```
posts = Post.objects.filter(published=True)
```

Exclude

```
posts = Post.objects.exclude(published=False)
```

Update

```
Post.objects.filter(id=1).update(
    title="Updated Title"
)
```

Delete

```
Post.objects.filter(id=1).delete()
```

Field Lookups

```
Post.objects.filter(title__icontains="django")
Post.objects.filter(title__startswith="Django")
Post.objects.filter(id__gt=5)
Post.objects.filter(id__gte=5)
Post.objects.filter(id__lt=10)
Post.objects.filter(id__in=[1, 2, 3])
```

Ordering

```
Post.objects.order_by("title")
Post.objects.order_by("-created_at")
```

Limit Results

```
Post.objects.all()[:10]
```

Count and Existence

```
Post.objects.count()
Post.objects.filter(published=True).exists()
```

Q Objects

```
from django.db.models import Q

posts = Post.objects.filter(
    Q(title__icontains="python")
    | Q(content__icontains="python")
)
```

F Expressions

```
from django.db.models import F

# Example for a model containing a views_count field.
Post.objects.filter(pk=1).update(
    views_count=F("views_count") + 1
)
```

`F()` expressions allow database-side operations without first reading the current value into Python.

---

12. CRUD Operations

CRUD means Create, Read, Update, and Delete.

Create

```
post = Post.objects.create(
    title="First Post",
    content="Hello Django",
)
```

Read

```
posts = Post.objects.all()
```

Update

```
post = Post.objects.get(pk=1)
post.title = "Updated Post"
post.save()
```

Delete

```
post = Post.objects.get(pk=1)
post.delete()
```

Use `get_object_or_404()` when a missing record should result in an HTTP 404 response.

---

13. Django Forms and ModelForms

Regular Form

`core/forms.py`

```
from django import forms


class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(
```

```
        widget=forms.Textarea
    )
```

ModelForm

```
from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["title", "content", "published"]
```

Process a Form

```
from django.shortcuts import render, redirect
from .forms import PostForm


def create_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("post_list")
    else:
        form = PostForm()

    return render(
        request,
        "posts/form.html",
        {"form": form},
    )
```

Form Template

```
<form method="post">
    {% csrf_token %}

    {{ form.as_p }}
```

```
    <button type="submit">Save</button>
</form>
```

Important Rules

- Use `form.is_valid()` before accessing cleaned data.
- Use `form.cleaned_data` for validated values.
- Include `{% csrf_token %}` in internal POST forms.
- Use explicit `ModelForm` fields rather than blindly exposing every model field.

---

14. Django Admin

Django provides an administrative interface for managing application data.

Register a Model

`core/admin.py`

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = (
        "id",
        "title",
        "published",
        "created_at",
    )
    list_filter = ("published",)
    search_fields = ("title", "content")
    ordering = ("-created_at",)
```

Create an administrator:

```
python manage.py createsuperuser
```

Open:

```
http://127.0.0.1:8000/admin/
```

---

15. Authentication and Authorization

Authentication identifies the user. Authorization determines what the user is allowed to do.

Login

```
from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect


def login_view(request):
    if request.method == "POST":
        username = request.POST.get("username", "")
        password = request.POST.get("password", "")

        user = authenticate(
            request,
            username=username,
            password=password,
        )

        if user is not None:
            login(request, user)
            return redirect("home")

    return render(request, "accounts/login.html")
```

For production applications, use Django forms to validate login input and return appropriate errors.

Logout

```
from django.contrib.auth import logout
from django.shortcuts import redirect


def logout_view(request):
    if request.method == "POST":
        logout(request)

    return redirect("home")
```

A logout action should use POST rather than a state-changing GET request.

LoginRequiredMixin

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView
from .models import Post


class PrivatePostListView(LoginRequiredMixin, ListView):
    model = Post
    template_name = "posts/list.html"
```

LoginRequired Decorator

```
from django.contrib.auth.decorators import login_required


@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

User Registration

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth import get_user_model


class RegistrationForm(UserCreationForm):
    class Meta:
        model = get_user_model()
        fields = ["username", "email"]
```

`UserCreationForm` provides password fields and password validation.

Object Ownership

Checking authentication alone does not ensure that a user owns a particular post.

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, redirect
from django.views.decorators.http import require_POST
from .models import Post


@login_required
@require_POST
def delete_post(request, pk):
```

```
    post = get_object_or_404(
        Post,
        pk=pk,
        author=request.user,
    )

    post.delete()
    return redirect("post_list")
```

This example assumes that `Post` has an `author` ForeignKey to the user model.

---

16. Sessions and Cookies

Sessions

Sessions store user-specific state on the server, with a session identifier typically held in a cookie.

```
def visit_counter(request):
    count = request.session.get("count", 0)
    count += 1
    request.session["count"] = count

    return HttpResponse(f"Visits: {count}")
```

Delete a Session Key

```
del request.session["count"]
```

Cookies

Set a cookie:

```
from django.http import HttpResponse


def set_cookie(request):
    response = HttpResponse("Cookie set")

    response.set_cookie(
        "theme",
        "dark",
        max_age=3600,
        httponly=True,
        secure=True,
```

```
        samesite="Lax",
    )

    return response
```

`secure=True` requires HTTPS for the cookie to be sent by browsers.

Cookies are client-side values and should not be trusted for authorization decisions.

---

17. Django Middleware

Middleware processes requests and responses around Django's view handling.

Common built-in middleware includes:

- `SecurityMiddleware`
- `SessionMiddleware`
- `CommonMiddleware`
- `CsrfViewMiddleware`
- `AuthenticationMiddleware`
- `MessageMiddleware`

Custom Middleware

```
class SimpleMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        # Before the view
        response = self.get_response(request)
        # After the view
        return response
```

Register it in `MIDDLEWARE`:

```
MIDDLEWARE = [
    # Existing Django middleware
    "core.middleware.SimpleMiddleware",
]
```

Middleware order matters. Preserve the required order of Django's built-in middleware.

---

18. File Uploads

Model

```
from django.db import models


class Document(models.Model):
    title = models.CharField(max_length=100)
    file = models.FileField(upload_to="documents/")
```

Settings

```
MEDIA_URL = "media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Form

```
from django import forms
from .models import Document


class DocumentForm(forms.ModelForm):
    class Meta:
        model = Document
        fields = ["title", "file"]
```

View

```
from django.shortcuts import render, redirect
from .forms import DocumentForm


def upload_document(request):
    if request.method == "POST":
        form = DocumentForm(
            request.POST,
            request.FILES,
        )

        if form.is_valid():
            form.save()
            return redirect("upload_document")
    else:
        form = DocumentForm()

    return render(
```

```
        request,
        "documents/upload.html",
        {"form": form},
    )
```

Template

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Upload</button>
</form>
```

Validate file types and sizes, and configure production media storage securely. Do not assume a filename extension alone makes a file safe.

---

19. Pagination

```
from django.core.paginator import Paginator
from django.shortcuts import render
from .models import Post


def post_list(request):
    paginator = Paginator(
        Post.objects.order_by("-created_at"),
        10,
    )

    page_obj = paginator.get_page(
        request.GET.get("page")
    )

    return render(
        request,
        "posts/list.html",
        {"page_obj": page_obj},
    )
```

Template

```
{% for post in page_obj %}
    <h2>{{ post.title }}</h2>
{% empty %}
    <p>No posts found.</p>
{% endfor %}

{% if page_obj.has_previous %}
    <a href="?page={{ page_obj.previous_page_number }}">
        Previous
    </a>
{% endif %}

<span>
    Page {{ page_obj.number }} of {{ page_obj.paginator.num_pages }}
</span>

{% if page_obj.has_next %}
    <a href="?page={{ page_obj.next_page_number }}">
        Next
    </a>
{% endif %}
```

---

20. Messages Framework

Django's messages framework displays one-time notifications.

View

```
from django.contrib import messages
from django.shortcuts import redirect


def success_view(request):
    messages.success(request, "Operation successful!")
    return redirect("home")
```

Template

```
{% if messages %}
    {% for message in messages %}
        <p>{{ message }}</p>
    {% endfor %}
{% endif %}
```

Message levels include debug, info, success, warning, and error.

---

21. Django Signals

Signals allow components to notify other components when certain events occur.

Example

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth import get_user_model


@receiver(post_save, sender=get_user_model())
def user_saved(sender, instance, created, **kwargs):
    if created:
        print("A new user was created")
```

Register signal handlers reliably, commonly through an app's `apps.py` and `ready()` method.

Signals are useful for decoupled event handling, but explicit service functions are often easier to understand for core business workflows.

---

22. Generic Class-Based Views

Django provides generic views for common tasks.

ListView

```
from django.views.generic import ListView
from .models import Post


class PostListView(ListView):
    model = Post
    template_name = "posts/list.html"
    context_object_name = "posts"
    paginate_by = 10
```

DetailView

```
from django.views.generic import DetailView
from .models import Post
```

```
class PostDetailView(DetailView):
    model = Post
    template_name = "posts/detail.html"
```

CreateView

```
from django.urls import reverse_lazy
from django.views.generic import CreateView
from .models import Post


class PostCreateView(CreateView):
    model = Post
    fields = ["title", "content"]
    template_name = "posts/form.html"
    success_url = reverse_lazy("post_list")
```

UpdateView

```
from django.views.generic import UpdateView
from .models import Post


class PostUpdateView(UpdateView):
    model = Post
    fields = ["title", "content"]
    template_name = "posts/form.html"
    success_url = reverse_lazy("post_list")
```

DeleteView

```
from django.views.generic import DeleteView
from .models import Post


class PostDeleteView(DeleteView):
    model = Post
    template_name = "posts/confirm_delete.html"
    success_url = reverse_lazy("post_list")
```

Protect create, update, and delete views with authentication and appropriate ownership or permission checks.

---

23. Query Optimization

select_related()

Use `select_related()` for ForeignKey and OneToOne relationships.

```
posts = Post.objects.select_related("author")
```

prefetch_related()

Use `prefetch_related()` for ManyToMany and reverse relationships.

```
posts = Post.objects.prefetch_related("tags")
```

These examples assume that the relevant model relationships exist.

only()

```
Post.objects.only("id", "title")
```

defer()

```
Post.objects.defer("content")
```

Explain a Query

```
print(Post.objects.filter(published=True).explain())
```

Use Django Debug Toolbar or query logging to identify expensive database operations before optimizing.

---

24. Relationships Between Models

One-to-Many

```
from django.conf import settings
from django.db import models


class Post(models.Model):
    author = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="posts",
    )
    title = models.CharField(max_length=200)
```

One user can have many posts.

One-to-One

```
class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name="profile",
    )
    bio = models.TextField(blank=True)
```

One user has at most one associated profile through this relationship.

Many-to-Many

```
class Tag(models.Model):
    name = models.CharField(max_length=50)


class Article(models.Model):
    title = models.CharField(max_length=200)
    tags = models.ManyToManyField(
        Tag,
        related_name="articles",
        blank=True,
    )
```

One article can have multiple tags, and one tag can belong to multiple articles.

---

25. Django REST Framework (DRF)

Install DRF:

```
pip install djangorestframework
```

Add it to `INSTALLED_APPS`:

```
INSTALLED_APPS = [
    # Existing apps
    "rest_framework",
]
```

Serializer

`core/serializers.py`

```
from rest_framework import serializers
from .models import Post


class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post
        fields = [
            "id",
            "title",
            "content",
            "published",
            "created_at",
        ]
        read_only_fields = ["id", "created_at"]
```

API View

`core/views.py`

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .models import Post
from .serializers import PostSerializer


@api_view(["GET", "POST"])
def post_api(request):
    if request.method == "GET":
        posts = Post.objects.all()
        serializer = PostSerializer(posts, many=True)

        return Response(serializer.data)

    serializer = PostSerializer(data=request.data)

    if serializer.is_valid():
        serializer.save()

        return Response(
            serializer.data,
            status=status.HTTP_201_CREATED,
        )
```

```
    return Response(
        serializer.errors,
        status=status.HTTP_400_BAD_REQUEST,
    )
```

API URL

```
from django.urls import path
from .views import post_api

urlpatterns = [
    path("api/posts/", post_api, name="post_api"),
]
```

This basic example permits POST requests unless you add authentication and permissions. Do not expose write operations publicly without deciding who is allowed to use them.

ModelViewSet

```
from rest_framework.viewsets import ModelViewSet
from rest_framework.permissions import IsAuthenticatedOrReadOnly
from .models import Post
from .serializers import PostSerializer


class PostViewSet(ModelViewSet):
    queryset = Post.objects.all()
    serializer_class = PostSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

Register the ViewSet with a DRF router.
`IsAuthenticatedOrReadOnly` allows authenticated users to write but does not enforce ownership of individual posts. Add object-level permission checks when needed.

---

26. API Authentication and Permissions

Authentication Options

- Session authentication
- Basic authentication
- Token authentication

- JWT authentication through an appropriate package
- OAuth-based authentication through an appropriate provider

Permission Examples

```
from rest_framework.permissions import IsAuthenticated

permission_classes = [IsAuthenticated]
```

Allow authenticated users to read and write, with appropriate ownership restrictions where required.

```
from rest_framework.permissions import IsAuthenticatedOrReadOnly

permission_classes = [IsAuthenticatedOrReadOnly]
```

Use throttling, validation, pagination, and appropriate authentication policies for production APIs.

---

27. Django Security Best Practices

- Keep `DEBUG = False` in production.
- Store `SECRET_KEY` in an environment variable or secret manager.
- Configure `ALLOWED_HOSTS`.
- Use HTTPS.
- Keep CSRF protection enabled.
- Use Django's password hashing system.
- Validate and sanitize incoming data appropriately.
- Enforce authentication and authorization.
- Check object ownership for private resources.
- Avoid exposing sensitive information in error responses.
- Validate uploaded files.
- Keep dependencies updated.
- Configure secure cookies.
- Back up important data.
- Review production logs and permissions.

Example Environment Variables

```
import os

SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
DEBUG = os.environ.get("DJANGO_DEBUG", "False").lower() == "true"

ALLOWED_HOSTS = os.environ.get(
    "DJANGO_ALLOWED_HOSTS",
    "",
).split(",")
```

Configure the environment variables explicitly before starting the application.

---

28. Django Testing

Test a View

`core/tests.py`

```
from django.test import TestCase
from django.urls import reverse


class HomeViewTests(TestCase):
    def test_home_returns_200(self):
        response = self.client.get(reverse("home"))

        self.assertEqual(response.status_code, 200)
```

Test a Model

```
from django.test import TestCase
from .models import Post


class PostModelTests(TestCase):
    def test_post_string_representation(self):
        post = Post(
            title="Test Post",
            content="Test content",
        )

        self.assertEqual(str(post), "Test Post")
```

Run tests:

```
python manage.py test
```

Test successful requests, validation failures, authentication, authorization, and important edge cases.

---

29. Deployment

Production Checklist

1. Set `DEBUG = False`.
2. Configure `SECRET_KEY` securely.
3. Set `ALLOWED_HOSTS`.
4. Configure the production database.
5. Apply migrations.
6. Configure static files.
7. Configure media storage.
8. Set up HTTPS.
9. Configure logging.
10. Run Django's deployment checks.

```
python manage.py check --deploy
python manage.py migrate
python manage.py collectstatic
```

WSGI and ASGI

- WSGI is the traditional synchronous Python web server interface.
- ASGI supports asynchronous applications and protocols.

Choose a suitable application server and hosting platform for the project's requirements.

---

30. Django Interview Questions

Beginner Questions

1. What is Django?

Django is a Python web framework for building secure and maintainable web applications.

2. What is MVT?

MVT means Model, View, and Template.

3. What is a Django app?

An app is a modular component of a Django project that handles a particular set of features.

4. What is ORM?

ORM maps Python objects and model operations to database operations.

5. What are migrations?

Migrations record and apply database schema changes.

6. What is a QuerySet?

A QuerySet represents a collection of database queries and results. QuerySets are generally lazy until evaluated.

7. What is middleware?

Middleware processes requests and responses globally around view execution.

8. What is CSRF?

Cross-Site Request Forgery is an attack that tricks a user's browser into submitting an unwanted request. Django provides CSRF protection.

9. What is `get_object_or_404()`?

It retrieves an object or raises an HTTP 404 response when the object does not exist.

10. What is `reverse()`?

It resolves a named URL pattern into a URL string.

Intermediate Questions

11. FBV vs CBV?

FBVs use functions. CBVs use classes, inheritance, and mixins.

12. `get()` vs `filter()`?

`get()` expects exactly one result. `filter()` returns a QuerySet.

13. `select_related()` vs `prefetch_related()`?

`select_related()` joins related single-valued relationships. `prefetch_related()` fetches related objects separately and joins them in Python.

14. `blank=True` vs `null=True`?

`blank=True` affects form validation. `null=True` affects database storage.

15. What is a signal?

A signal notifies registered receivers when a specific event occurs.

16. What is `transaction.atomic()`?

It makes database operations inside a transaction succeed or roll back together.

17. What is `Q()`?

It builds complex query conditions, including OR and NOT logic.

18. What is `F()`?

It references a database field within a query or update.

19. What is a custom manager?

A model manager provides a reusable interface for querying model data.

20. What is `on_delete`?

It defines how related objects are handled when the referenced object is deleted.

Advanced Questions

21. How do you prevent N+1 queries?

Use appropriate eager loading with `select_related()` or `prefetch_related()` and inspect actual query behavior.

22. How do you secure a REST API?

Use suitable authentication, permissions, validation, throttling, HTTPS, and object-level authorization.

23. How do you optimize Django performance?

Profile first, optimize queries, add suitable indexes, cache expensive results, paginate responses, and monitor production performance.

24. How do you handle database transactions?

Use `transaction.atomic()` when a group of database changes must be committed or rolled back together.

25. How do you prevent users from editing another user's objects?

Filter the object by its owner or implement and enforce appropriate object-level permissions.

26. What is a custom user model?

A user model that customizes Django's user representation. If customization is anticipated, define it early and configure `AUTH_USER_MODEL` before the initial migrations.

27. What is caching?

Caching stores reusable results to reduce repeated computation or database work.

28. What is idempotency?

An operation is idempotent when repeating it has the same intended effect as performing it once.

29. How do you handle background tasks?

Use a suitable task queue and worker system when work should run outside the request-response cycle.

30. What is the difference between WSGI and ASGI?

WSGI targets the traditional synchronous application interface; ASGI supports asynchronous execution and additional protocols.

---

31. Practice Projects

Beginner

1. Personal Portfolio Website
2. Blog Application
3. Todo List Application
4. Contact Management System
5. Student Management System

Intermediate

1. Authentication System
2. Library Management System
3. E-commerce Backend
4. Job Portal
5. REST API for a Blog
6. Expense Tracker

Advanced

1. Multi-vendor E-commerce Platform
2. Learning Management System
3. Real-time Chat Application
4. Inventory Management API
5. Booking and Reservation System
6. SaaS Application with Role-Based Access

Recommended Blog Project Structure

```
blog_project/
├── manage.py
├── config/
│   ├── settings.py
│   └── urls.py
├── accounts/
├── posts/
│   ├── models.py
│   ├── views.py
│   ├── urls.py
│   ├── forms.py
│   ├── admin.py
│   └── tests.py
├── templates/
│   ├── base.html
│   ├── accounts/
│   └── posts/
└── static/
```

Blog Project Features

- User registration
- Login and logout
- Create posts
- List posts
- View post details
- Update posts
- Delete posts
- Pagination
- Search
- Categories and tags
- Comments
- Image uploads
- User ownership checks
- Admin management
- REST API
- Automated tests

---

32. Recommended Learning Roadmap

Phase 1: Fundamentals

Learn Python, HTTP, HTML, CSS, and basic SQL.

Phase 2: Django Basics

Learn project setup, apps, URLs, views, templates, models, and migrations.

Phase 3: CRUD and Forms

Build complete CRUD applications using forms, validation, and ORM operations.

Phase 4: Authentication

Implement registration, login, logout, permissions, and object ownership.

Phase 5: Advanced Django

Learn generic views, middleware, signals, pagination, file uploads, transactions, and query optimization.

Phase 6: Django REST Framework

Learn serializers, API views, ViewSets, routers, authentication, permissions, and pagination.

Phase 7: Production

Learn testing, environment configuration, deployment, database optimization, security, and logging.

Phase 8: Interview Preparation

Practice coding problems, explain project architecture, and prepare to discuss trade-offs and security decisions.

Final Advice

For every topic:

1. Understand the concept.
2. Write the code yourself.
3. Test successful and failed cases.
4. Build a small feature using the concept.
5. Explain the solution in your own words.
6. Add automated tests.
7. Review security and performance.

Recommended order: Django fundamentals → ORM and CRUD → authentication → generic views → Django REST Framework → testing → deployment.

Use the official Django documentation as the reference for version-specific behavior: https://docs.djangoproject.com/